

To get list of invoice items that belongs to invoice, then we use, Include-method:

```
[HttpGet]
public async Task<ActionResult<IEnumerable<Invoice>>> GetInvoices(
    int page = 1, int pageSize = 10, InvoiceStatus? status = null
)
{
    return await _context
        .Invoices
        .AsQueryable()
        .Include(x => x.InvoiceItems)
        .Where(t1 => t1.Status == status)
        .OrderByDescending(x => x.InvoiceDate)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .ToListAsync();
}
```

Note: The `Include()` method is a convenient way to include dependent entities. However, it may cause performance issues **when the collection of dependent entities is large**. For example, a post may have hundreds or thousands of comments. It is not a good idea to include all comments in the query result for a list page. In this case, it is not necessary to include dependent entities in the query.

3. Note that the query includes Invoice data in each row of the result. For some scenarios, it may cause a so-called **Cartesian explosion** problem, which means the amount of duplicated data in the result is too large and may cause performance issues. In this case, we can split the queries into two steps. First, we query the invoices, and then we query the invoice items. You need to use the `AsSplitQuery()` method as follows:

```
[HttpGet]
public async Task<ActionResult<IEnumerable<Invoice>>>
GetInvoices(int page = 1, int pageSize = 10,
    InvoiceStatus? status = null)
{
    // Omitted for brevity
    return await context.Invoices
        .Include(x => x.InvoiceItems)
        .Where(x => status == null || x.Status == status)
        .OrderByDescending(x => x.InvoiceDate)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .AsSplitQuery()
        .ToListAsync();
}
```

4. You can also configure the default query-splitting behavior globally by using the `UseQuerySplittingBehavior()` method in the `OnConfiguring()` method of your `DbContext` class. The following code shows how to configure the default query splitting behavior to `SplitQuery` in the `SampleDbContext` class:

```
protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
{
    base.OnConfiguring(optionsBuilder);
    optionsBuilder.UseSqlServer(_configuration.
GetConnectionString("DefaultConnection"),
        b => b.UseQuerySplittingBehavior(QuerySplittingBehavior.
SplitQuery));
}
```

In this case, you don't need to use the `AsSplitQuery()` method in your LINQ queries. If you want to execute a specific query in a single query, you can use the `AsSingleQuery()` method like this:

```
[HttpGet]
public async Task<ActionResult<IEnumerable<Invoice>>>
GetInvoices(int page = 1, int pageSize = 10,
    InvoiceStatus? status = null)
{
    // Omitted for brevity

    return await _context.Invoices
        .Include(x => x.InvoiceItems)
        .Where(x => status == null || x.Status == status)
        .OrderByDescending(x => x.InvoiceDate)
        .Skip((page - 1) * pageSize)
        .Take(pageSize)
        .AsSingleQuery()
        .ToListAsync();
}
```

Activ
Go to S

However, split queries may cause other issues. For example, **multiple queries increase the number of round trips to the database**. In addition, if another thread modifies the data between the two queries, **the result may be inconsistent**. Therefore, you should consider the pros and cons of split queries to fit our scenarios.

```

public class PostConfiguration : IEntityTypeConfiguration<Post>
{
    public void Configure(EntityTypeBuilder<Post> builder)
    {
        builder.ToTable("Posts");
        // Omitted for brevity
        builder.Property(p => p.CategoryId).
            HasColumnName("CategoryId");
        builder.HasOne(p => p.Category)
            .WithMany(c => c.Posts)
            .HasForeignKey(p => p.CategoryId)
            .OnDelete(DeleteBehavior.ClientSetNull);
    }
}

```

In the CategoriesController class, you can find the DeleteCategory() method. It is similar to the DeleteInvoice() method in the InvoicesController class. The only difference is that we need to remove the relationship between the category and the blog posts before deleting the category. The following code shows how to remove the relationship between the category and the blog posts:

```

var category = await context.Categories.Include(x => x.Posts).
    SingleOrDefaultAsync(x => x.Id == id);
if (category == null)
{
    return NotFound();
}
category.Posts.Clear();
// Or you can update the posts to set the category to null
// foreach (var post in category.Posts)
// {
//     post.Category = null;
// }
context.Categories.Remove(category);
await context.SaveChangesAsync();

```

Activate W
Go to Settings

You can clear the Posts property of the category entity, or you can update the Category property of the blog posts to set it to null. In this way, the CategoryId property of the blog posts will be set to null when the category is deleted. Also, it is required to load related entities using the Include method, because EF Core needs to track the changes of related entities as well.

To create many to many relationship between two models:

```
namespace JLokaTestEFCore.Models
{
    public class Movie
    {
        public Guid Id { get; set; }
        public string Title { get; set; } = string.Empty;
        public string? Description { get; set; }
        public int ReleaseYear { get; set; }
        public List<Actor> Actors { get; set; } = new List<Actor>();
    }
}
```

```
namespace JLokaTestEFCore.Models
{
    public class Actor
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public List<Movie> Movies { get; set; } = new List<Movie>();
    }
}
```

To create specific configurations for many to many relationship:

```
using JLokaTestEFCore.enums;
using JLokaTestEFCore.Models;
using Microsoft.EntityFrameworkCore;

namespace JLokaTestEFCore.Data
{
    public static class MovieModelCreatingExtensions
    {
        public static void ConfigureMovie(this ModelBuilder builder)
        {
            builder.Entity<Movie>(b =>
            {
                b.ToTable("Movies");
                b.HasKey(i => i.Id);
                b.Property(p => p.Id).HasColumnName("Id");
                b.Property(p =>
                p.Title).HasColumnName(nameof(Movie.Title)).HasMaxLength(255).IsRequired();
                b.Property(p =>
                p.Description).HasColumnName(nameof(Movie.Description));
                b.Property(p =>
                p.ReleaseYear).HasColumnName(nameof(Movie.ReleaseYear)).IsRequired();
                b.HasMany(m => m.Actors).WithMany(a =>
                a.Movies).UsingEntity<MovieActor>(

                    j => j.HasOne(ma => ma.Actor).WithMany(a =>
                    a.MovieActors).HasForeignKey(ma => ma.ActorId),

                    j => j.HasOne(ma => ma.Movie).WithMany(m =>
                    m.MovieActors).HasForeignKey(ma => ma.MovieId),

                    j =>
                    {
                        j.Property(ma =>
                        ma.UpdateTime).HasColumnName(nameof(MovieActor.UpdateTime))
                        .HasDefaultValueSql("CURRENT_TIMESTAMP");
                        j.HasKey(ma => new { ma.MovieId, ma.ActorId});
                    }
                );
            });
        }
    }
}
```

Another way (before EF Core 5.0) to configure a many-to-many relationship is to use the join entity to represent two separate one-to-many relationships. Here is an example of configuring a many-to-many relationship for the Movie and Actor entities:

```
public class MovieActorsConfiguration :
    IEntityTypeConfiguration<MovieActor>
{
    public void Configure(EntityTypeBuilder<MovieActor> builder)
    {
        builder.ToTable("MovieActors");
        builder.HasKey(sc => new { sc.MovieId, sc.ActorId });
        builder.HasOne(sc => sc.Actor)
            .WithMany(s => s.MovieActors)
            .HasForeignKey(sc => sc.ActorId);
        builder.HasOne(sc => sc.Movie)
            .WithMany(c => c.MovieActors)
            .HasForeignKey(sc => sc.MovieId);
    }
}
```

In a many-to-many relationship, such as Movie and Actor, we may need to get the actors of a movie or get the movies of an actor. So, we need to expose both entities through the REST API. You can create two controllers using the following commands:

Ex: `dotnet aspnet-codegenerator controller -name MoviesController -api -outDir Controllers --model Movie --dataContext InvoiceDbContext -async -actions`

Ex: `dotnet aspnet-codegenerator controller -name ActorsController -api -outDir Controllers --model Actor --dataContext InvoiceDbContext -async -actions`

To create unique index:

```
using JLokaTestEFCore.enums;
using JLokaTestEFCore.Models;
using Microsoft.EntityFrameworkCore;

namespace JLokaTestEFCore.Data
{
    public static class MovieModelCreatingExtensions
    {
        public static void ConfigureMovie(this ModelBuilder builder)
        {
            builder.Entity<Movie>(b =>
            {
                b.ToTable("Movies");
                b.HasKey(i => i.Id);
                b.Property(p => p.Id).HasColumnName("Id");
                b.Property(p =>
                p.Title).HasColumnName(nameof(Movie.Title)).HasMaxLength(255).IsRequired();
                b.Property(p =>
                p.Description).HasColumnName(nameof(Movie.Description));
                b.Property(p =>
                p.ReleaseYear).HasColumnName(nameof(Movie.ReleaseYear)).IsRequired();
                b.HasMany(m => m.Actors).WithMany(a =>
                a.Movies).UsingEntity<MovieActor>(
                    j => j.HasOne(ma => ma.Actor).WithMany(a =>
                    a.MovieActors).HasForeignKey(ma => ma.ActorId),
                    j => j.HasOne(ma => ma.Movie).WithMany(m =>
                    m.MovieActors).HasForeignKey(ma => ma.MovieId),
                    j =>
                    {
                        j.Property(ma =>
                        ma.UpdateTime).HasColumnName(nameof(MovieActor.UpdateTime))
                        .HasDefaultValueSql("CURRENT_TIMESTAMP");
                        j.HasKey(ma => new { ma.MovieId, ma.ActorId});
                    }
                );
                b.HasIndex(p => p.Title).IsUnique();
            });
        }
    }
}
```

```

[HttpPost("{id}/movies/{movieId}")]
public async Task<IActionResult> AddMovie(Guid id, Guid movieId)
{
    if(_context.actors == null)
    {
        return NotFound("Actors is null");
    }

    var actor = await _context.actors.Include(x => x.movies).SingleOrDefaultAsync(x
=> x.Id == id);

    if(actor == null)
    {
        return NotFound($"Actor with id: {id} not found");
    }

    var movie = await _context.movies.FindAsync(movieId);
    if(movie == null)
    {
        return NotFound($"Movie with id: {id} not found");
    }

    if(actor.movies.Any(x => x.Id == movieId))
    {
        return Problem($"Movie with id: {movieId} already exists for actor with id:
{id}");
    }

    // here we should check for existing of movies for actor before adding
    actor.movies.Add(movie);
    await _context.SaveChangesAsync();
    return CreatedAtAction("GetActor", new { Id = actor.Id }, actor);
}

[HttpGet("{id}/movies")]
public async Task<IActionResult> GetMovies(Guid id)
{
    if(_context.actors == null)
    {
        return NotFound("Actors is null");
    }

    var actor = await _context.actors.Include(x => x.movies).SingleOrDefaultAsync(x
=> x.Id == id);

    if(actor == null)
    {
        return NotFound($"No actor with id: {id}");
    }
    // here we must make the response with dto
    return Ok(actor.movies);
}

[HttpDelete("{id}/movies/{movieId}")]
public async Task<IActionResult> DeleteMovie(Guid id, Guid movieId)
{
    if(_context.actors == null)

```

```

    {
        return NotFound("Actors is null");
    }

    var actor = await _context.Actors.Include(x => x.Movies).SingleOrDefaultAsync(x
=> x.Id == id);

    if(actor == null)
    {
        return NotFound($"No Actor with Id: {id}");
    }

    var movie = await _context.Movies.FindAsync(movieId);
    if(movie == null)
    {
        return NotFound($"No movie with id: {movieId}");
    }
    // Note that it does not delete the movie from the database; it just deletes
    // the relationship between the movie and the actor.
    actor.Movies.Remove(movie);
    await _context.SaveChangesAsync();

    return NoContent();
}

```
